REAL TIME CHAT APPLICATION

A PROJECT REPORT SUBMITTED TO

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

IN PARTIAL FULFILMENT OF THE REQUIREMENTS FOR THE

AWARD OF THE DEGREE OF

MASTER OF COMPUTER APPLICATIONS

BY

PERARASU B (RA2332241010206)

AJMAL M (RA2332241010184)

BILGATES M (RA2332241010223)

FAHEEM HM (RA2332241010229)

UNDER THE GUIDANCE OF

Dr. Ananthi Claral Mary.T, M.C.A, Ph.D

DEPARTMENT OF COMPUTER APPLICATIONS

FACULTY OF SCIENCE AND HUMANITIES

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

Kattankulathur – 603 203

Chennai, Tamil Nadu

NOVEMBER - 2024

BONAFIDE CERTIFICATE

This is to certify that the project report titled "**REAL TIME CHAT APPLICATION**" is a bonafide work carried out by **PERARASU B** (RA2132241010206) , **AJMAL M** (RA2332241010184) , **BILGATES M** (RA2332241010223), **FAHEEM HM** (RA2332241010229) under my supervision for the award of the Degree of Master of Computer Applications. To my knowledge the work reported herein is the original work done by these students.

Dr. T.Ananthi Claral Mary
Assistant Professor,
Department of Computer Applications
(GUIDE)

Dr. R.Jayashree
Associate Professor & Head,
Department of Computer Applications

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

With profound gratitude to the ALMIGHTY, I take this chance to thank the people who helped me to complete this project.

We take this as a right opportunity to say THANKS to my parents who are there to stand with me always with the words “YOU CAN”.

We are thankful to **Dr. T. R. Paarivendhar**, Chancellor, and **Prof. A. Vinay Kumar**, Pro Vice-Chancellor (SBL), SRM Institute of Science & Technology, who gave us the platform to establish me to reach greater heights.

We earnestly thank **Dr. A. Duraisamy**, Dean, Faculty of Science and Humanities, SRM Institute of Science & Technology, who always encourage us to do novel things.

A great note of gratitude to **Dr. S. Albert Antony Raj**, Deputy Dean, Faculty of Science and Humanities for his valuable guidance and constant Support to do this Project.

We express our sincere thanks to **Dr. R. Jayashree**, Associate Professor & Head, for her support to execute all incline in learning.

It is our delight to thank our project guide **Dr. Ananthi Claral Mary.T,** Assistant Professor, Department of Computer Applications, for her help, support, encouragement, suggestions, and guidance throughout the development phases of the project.

We convey our gratitude to all the faculty members of the department who extended their support through valuable comments and suggestions during the reviews.

Our gratitude to friends and people who are known and unknown to me who helped in carrying out this project work a successful one.

PERARASU B

AJMAL M

BILGATES M

FAHEEM HM

Dr.Ananthi Claral Mary T

Chat applical'ion

air

abstract

MCA

SRM Institute of Science & Technology

Document Details

Submission ID

trn:oid:::1:3059300730

Submission Date

Oct 29, 2024, 1:37 PM GMT+5:30

Download Date

Oct 29, 2024, 1:38 PM GMT+5:30

File Name

CHATBOT_ABSTRACT.pdf

File Size

75.0 KB

1 Page

300 Words

1,943 Characters

0% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

0 Not Cited or Quoted 0%
Matches with neither in-text citation nor quotation marks

0 Missing Quotations 0%
Matches that are still very similar to source material

0 Missing Citation 0%
Matches that have quotation marks, but no in-text citation

0 Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

Internet sources

0 Publications

0 Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Match Groups

0 Not Cited or Quoted 0%
Matches with neither in-text citation nor quotation marks

0 Missing Quotations 0%
Matches that are still very similar to source material

0 Missing Citation 0%
Matches that have quotation marks, but no in-text citation

0 Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

TABLE OF CONTENTS

ABSTRACT

This real-time chat application presents a cutting-edge solution for online communication, seamlessly blending an exceptional user experience with robust security and scalability. Developed using React for the front-end and Firebase for the backend, it incorporates Material-UI (MUI) components to create a modern and intuitive interface. The design prioritizes user-friendliness, ensuring that both tech-savvy individuals and those less familiar with technology can navigate the platform with ease. Security is a cornerstone of the application, providing users with secure access through Google Authentication integrated with Firebase. This feature allows for effortless login while safeguarding sensitive information, making users feel confident in the safety of their data. The combination of a secure authentication process and a focus on user privacy reinforces the application's commitment to protecting its users. Real-time tracking of active users further enhances the communication experience, enabling efficient conversation management. A dynamic user list allows participants to see who is online, fostering better interaction during chats. Additionally, a comprehensive message history feature ensures that users can easily revisit past conversations, maintaining context and continuity in their discussions, whether for personal chats or professional collaborations.

To optimize performance, the application utilizes Firebase Firestore and Realtime Database for storing messages and user information. This robust infrastructure enables fast data retrieval and storage, allowing for smooth real-time synchronization across devices. Such capabilities ensure that users experience minimal latency, making conversations feel fluid and immediate, which is crucial for effective communication. In summary, this real-time chat application redefines online communication by offering a blend of simplicity, security, and speed. Its scalable architecture is well-suited for a variety of use cases, from casual personal chats to more complex professional interactions. The combination of innovative technology and user-centric design makes it an ideal platform for anyone seeking a reliable and engaging way to connect with others.

CHAPTER 1

1.1 INTRODUCTION

This real-time chat application presents a cutting-edge solution for online communication, seamlessly blending exceptional user experience with robust security and scalability. Developed using React for the front end and Firebase for the backend, it incorporates Material-UI (MUI) components to create a modern and intuitive interface. The design prioritizes user-friendliness, ensuring that both tech-savvy individuals and those less familiar with technology can navigate the platform with ease.

Security is a cornerstone of the application, providing users with secure access through Google Authentication integrated with Firebase. This feature allows for effortless login while safeguarding sensitive information, making users feel confident in the safety of their data. The combination of a secure authentication process and a focus on user privacy reinforces the application's commitment to protecting its users. Real-time tracking of active users further enhances the communication experience, enabling efficient conversation management.

To optimize performance, the application utilizes Firebase Firestore and Realtime Database for storing messages and user information. This robust infrastructure enables fast data retrieval and storage, allowing for smooth real-time synchronization across devices. Such capabilities ensure that users experience minimal latency, making conversations feel fluid and immediate, which is crucial for effective communication. In summary, this real-time chat application redefines online communication by offering a blend of simplicity, security, and speed, making it an ideal platform for reliable and engaging connections.

1.2 PURPOSE AND OBJECTIVES

PURPOSE:

The purpose of this real-time chat application is to provide a cutting-edge platform for online communication that prioritizes user experience, security, and scalability. By leveraging modern technology, it aims to facilitate seamless interactions among users, regardless of their technical expertise.

OBJECTIVES:

1. Enhance User Experience:

To create an intuitive and user-friendly interface using React and Material-UI (MUI), ensuring that both tech-savvy individuals and those less familiar with technology can navigate the platform with ease.

2. Ensure Robust Security:

To implement secure access through Google Authentication, safeguarding user data and instilling confidence in users regarding the safety of their information.

3. Facilitate Efficient Communication:

To enable real-time tracking of active users and manage conversations effectively, enhancing interaction quality during chats.

4. Optimize Performance:

To utilize Firebase Firestore and Realtime Database for fast data retrieval and storage, ensuring smooth real-time synchronization and minimal latency for a fluid communication experience.

5. Support Diverse Use Cases:

To provide a scalable architecture that accommodates a variety of communication needs, from casual personal chats to complex professional interactions, making the platform versatile and adaptable.

1.3 PROJECT SCOPE:

1. Overview of the Application:

This project encompasses the development of a real-time chat application designed for online communication. The application aims to deliver an exceptional user experience while ensuring robust security and scalability, making it suitable for a wide range of users and use cases.

2. Key Features:

User-Friendly Interface:

Development of a modern interface using React and Material-UI (MUI) to ensure ease of navigation for all users, regardless of their technical background.

Secure Authentication:

Implementation of Google Authentication integrated with Firebase to provide secure login and protect user data, reinforcing user trust and privacy.

Real-Time User Tracking:

Incorporation of features that allow real-time tracking of active users, enhancing interaction quality and enabling effective conversation management.

Message History and Retrieval:

Development of a comprehensive message history feature that allows users to easily revisit past conversations, maintaining context in discussions.

3. Technical Infrastructure:

Backend Development:

Utilization of Firebase as the backend infrastructure to manage data storage and retrieval, ensuring efficient performance and real-time synchronization across devices.

Database Management:

Use of Firebase Firestore and Realtime Database for storing messages and user information, allowing for minimal latency and smooth communication experiences.

4. Performance Optimization:

Focus on optimizing the application's performance to ensure fast data retrieval and seamless real-time interactions, making conversations feel immediate and engaging.

5. Target Audience:

The application is designed to cater to a diverse audience, including casual users, professionals, and organizations seeking reliable and engaging communication solutions.

6. Future Enhancements:

Scope for future updates and enhancements based on user feedback, including potential integration with additional authentication methods, expanded features for conversation management, and support for multimedia messaging.

7. Project Timeline and Milestones:

Establish a clear timeline for development phases, including design, implementation, testing, and deployment, ensuring that the project stays on track and meets its objectives.

CHAPTER 2

SYSTEM CONFIGURATION

2.1 HARDWARE SPECIFICATION

COMPONENT	SPECIFICATION
Operating System	Windows 10 or newer, Mac OS X v10.7 or higher, Linux (Ubuntu)
Processor	2 GHz or higher
RAM	2.00 GB or above
Hard Disk	64 GB or more
Network	Ethernet connection (LAN) or a wireless adapter (Wi-Fi)

2.2 SOFTWARE SPECIFICATION

COMPONENT	SPECIFICATION
IDE	Visual Studio Code
Backend Framework	Firebase
Front End	HTML, CSS, React.js
Browser	Chrome

2.3 SOFTWARE DESCRIPTION:

1. Front-end:

Framework: React.js

React.js is a popular JavaScript library for building user interfaces, particularly single-page applications. It enables developers to create reusable UI components that can efficiently update and render based on changing data, enhancing performance and user experience.

UI Library: Material-UI (MUI)

Material-UI is a widely used React UI framework that provides a set of pre-designed components following Google's Material Design guidelines. This library allows developers to build modern, responsive, and visually appealing interfaces quickly, reducing the need for custom styling.

State Management: React Context and Redux

State management is crucial for managing application data and UI state. React Context provides a way to pass data through the component tree without props drilling, while Redux is a more robust state management solution that helps manage complex state interactions and provides a central store for the application's state.

Routing: React Router

React Router is a library that enables navigation among different components in a React application. It allows developers to define routes for various parts of the application, enabling a seamless user experience as users navigate between different views or pages.

2. Back-end:

Backend Service: Firebase

Firebase is a cloud-based platform that offers various services for building web and mobile applications. It provides a robust backend infrastructure, enabling developers to focus on application logic without worrying about server management.

Firebase Authentication: Google Auth

Firebase Authentication simplifies user authentication by providing secure login options. Integrating Google Auth allows users to sign in easily using their Google accounts, streamlining the login process while ensuring security.

Firestore & Realtime Database

Firestore and Realtime Database are Firebase's NoSQL database options. Firestore is designed for scalable applications with rich querying capabilities, while Realtime Database is optimized for real-time data synchronization. Both are used to store and sync messages, user information, and active status, allowing for dynamic, real-time interactions.

3. Real-time Communication:

Firestore Listeners

Firestore listeners are utilized to monitor changes in the database in real-time. By setting up listeners on specific collections or documents, the application can automatically update the UI whenever there are changes, such as new messages or user status updates, enhancing the communication experience.

4. Security:

Firestore Rules

Firestore security rules are essential for protecting user data and ensuring that only authorized users can access specific resources. These rules can be configured to restrict access based on user authentication status or specific criteria, thereby enforcing data security and privacy.

5. Deployment:

Frontend: Netlify

Netlify is a platform for deploying and hosting front-end applications. It provides a streamlined workflow for continuous deployment, enabling developers to easily deploy updates and manage their applications without extensive configuration.

Backend: Firebase Hosting

Firebase Hosting is a fast and secure way to host static and dynamic web applications. It provides a global content delivery network (CDN) to ensure quick loading times and automatic SSL certificates for secure connections, making it ideal for hosting the backend of the chat application.

6. Version Control:

GitHub for Code Management

GitHub is a popular platform for version control and collaboration. It allows developers to manage code repositories, track changes, and collaborate with others. Using GitHub enables efficient code management, version tracking, and facilitates teamwork through features like pull requests and issue tracking.

CHAPTER 3

SYSTEM ANALYSIS

3.1 EXISTING SYSTEM:

The existing Real time chat application systems often suffer from limitations in real-time communication, scalability, and secure data handling. Traditional chat platforms may not offer seamless user experiences due to slower backend services, lack of synchronization across devices, and security vulnerabilities. This inconsistency can frustrate users who expect smooth and instantaneous interactions. Many applications do not effectively integrate with Google Authentication, making the login process cumbersome and less secure. This can deter users from engaging fully with the platform, as complicated authentication methods often lead to abandoned logins. A simplified, secure login process is crucial for maintaining user trust and encouraging regular use of the application. Additionally, these systems may lack robust, real-time tracking of active users, which can hinder efficient communication and user engagement. Without this feature, users may feel disconnected from their contacts, impacting the overall chat experience and diminishing the platform's effectiveness as a communication tool.

Furthermore, traditional applications face significant challenges in scaling effectively as the user base grows. This often results in delayed message delivery and compromised performance, frustrating users and detracting from the overall experience. When performance lags, it can lead to decreased user satisfaction and increased churn as users search for more responsive alternatives. As a result, many existing chat solutions struggle to meet the evolving needs of their users, leaving them seeking more reliable and efficient alternatives. The demand for innovative solutions that address these shortcomings is growing, as users increasingly prioritize platforms that offer seamless communication, enhanced security, and a user-friendly experience.

3.2 PROPOSED SYSTEM:

Our proposed real-time chat application addresses the limitations of existing systems by introducing a modern, scalable platform with enhanced security and usability. The application is built using React for the front end and Firebase for the back end, leveraging Material-UI (MUI) components to ensure a user-friendly and visually appealing interface. This combination not only creates a seamless user experience but also facilitates easy navigation for users of all technical backgrounds.

Key Features:

1. Real-Time Communication:

The application utilizes Firebase Firestore listeners and the Realtime Database to provide instant updates to messages and user activity. This ensures that conversations remain synchronized across all devices, allowing users to communicate in real-time without delays. Whether on mobile or desktop, users will experience fluid interactions, making the chat experience more engaging and effective.

2. Enhanced Security:

Security is a top priority in our proposed system. By integrating Google Authentication with Firebase, we ensure that user data is safeguarded while simplifying the login process. This reduces friction for users when accessing the application. Additionally, Firestore Rules are implemented to restrict unauthorized access, protecting sensitive information and reinforcing user trust in the platform.

3. Efficient User Management:

The application features real-time tracking of active users, presented in a dynamic user list. This allows users to see who is online at any given moment, facilitating better communication. Users can also search for registered contacts quickly, streamlining the process of initiating conversations and enhancing overall engagement.

4. Scalability and Performance:

Built on Firebase's robust infrastructure, our application is designed to scale efficiently as the user base grows. This ensures fast data retrieval and reliable updates, maintaining performance even during peak usage times. Users can expect uninterrupted service, making the platform suitable for both casual chats and professional collaborations.

Overall, this proposed system not only enhances the user experience but also provides a secure, reliable, and scalable solution. It is designed to meet the needs of various users, from individuals seeking a simple chat tool to teams needing a collaborative platform for professional interactions. By addressing the limitations of existing chat applications, we aim to deliver a superior communication experience that adapts to the evolving demands of users.

3.3 FEASIBILITY STUDY:

1. Technical Feasibility:

The proposed real-time chat application is technically feasible, leveraging established technologies like React for the front end and Firebase for the back end. React is a widely adopted framework known for its performance and scalability, while Firebase provides a robust infrastructure that supports real-time data synchronization and secure user authentication. The use of Material-UI (MUI) enhances the user interface, ensuring it is visually appealing and user-friendly. The integration of Firebase Firestore and Realtime Database for instant updates further supports the technical viability of the application.

2. Operational Feasibility:

The application is designed to meet the operational needs of a diverse user base, from casual users to professional teams. With features like real-time user tracking, efficient user management, and enhanced security protocols, the platform can effectively facilitate communication in various contexts. The implementation of Google Authentication simplifies user onboarding, which is crucial for operational efficiency. Overall, the proposed system aligns well with user expectations for a modern chat application, making it operationally feasible.

3. Economic Feasibility:

From an economic perspective, the proposed system presents a sound investment. Utilizing Firebase can minimize infrastructure costs as it eliminates the need for extensive server management. Additionally, the scalability of the platform ensures that costs can be managed effectively as the user base grows. The focus on user engagement and retention through enhanced features may also lead to potential monetization opportunities, such as premium subscriptions or in-app purchases, further supporting the economic viability of the application.

4. Schedule Feasibility:

The timeline for developing the proposed chat application is realistic, given the technologies and frameworks involved. Development phases can be clearly outlined, including design, implementation, testing, and deployment. Utilizing established libraries and tools will expedite the development process, allowing for a quicker rollout while ensuring thorough testing to meet quality standards. The project can be completed within a reasonable timeframe, ensuring timely delivery to market.

5. Legal and Ethical Feasibility:

The proposed system adheres to legal and ethical standards, particularly regarding user data protection and privacy. By integrating Google Authentication and implementing Firestore Rules, the application prioritizes data security and compliance with regulations such as GDPR. Ensuring user consent for data usage and maintaining transparency about data handling practices further enhances the ethical feasibility of the application.

CHAPTER 4

SYSTEM DESIGN

The design of a real-time chat application involves several key components, each addressing different aspects of functionality, performance, security, and user experience. Below is an outline of the system design, including architecture, database schema, user interface, and security measures.

4.1 SYSTEM ARCHITECTURE

1. Architecture:

Client-Server Architecture:

- The application follows a client-server model where the front end communicates with the back end via API calls. The architecture can be visualized as follows:

Front End:

- Built with React.js, leveraging Material-UI for a responsive UI.
- Communicates with the backend via RESTful APIs or Firebase SDK.

Back End:

- Firebase serves as the backend, providing services like Firestore, Realtime Database, and Authentication. Manages data storage, user authentication, and real-time updates.

Real-Time Communication:

- Utilizes Firestore listeners and the Realtime Database to enable real-time messaging and user status updates.

2. Database Design:

Firestore Structure:

- The database schema will be designed to optimize data retrieval and real-time updates. Key collections may include.

Users Collection:

Document Fields:

- `userId`: Unique identifier for each user.
- `username`: Display name of the user.
- `email`: User's email address.
- `status`: Current online/offline status.
- `lastActive`: Timestamp of the last activity.

Messages Collection:

Document Fields:

- `messageId`: Unique identifier for each message.
- `senderId`: User ID of the message sender.
- `receiverId`: User ID of the message recipient (for direct messages) or `groupId` (for group chats).
- `content`: Text content of the message.
- `timestamp`: When the message was sent.
- `isRead`: Boolean to indicate if the message has been read.

Groups Collection (optional):

Document Fields:

- `groupId`: Unique identifier for each group chat.
- `groupName`: Name of the group.
- `members`: List of user IDs that are part of the group.
- `createdOn`: Timestamp of group creation.

3. User Interface Design:

UI Components:

The user interface should prioritize usability and accessibility. Key components include:

Login/Registration Screen:

- Google Authentication button for easy login.
- Fields for email and password for traditional login.

Chat Interface:

- Dynamic user list showing online users.
- Message input field at the bottom for composing messages.
- Display area for conversation history, showing timestamps and read status.

Group Chat Management:

- Interface for creating and managing group chats.
- Ability to add/remove members from groups.

Settings:

- User profile management (updating username, profile picture).
- Notification preferences and privacy settings.

4. Security Measures:

Data Protection:

Authentication:

- Use Firebase Authentication with Google Sign-In for secure login.

Firestore Security Rules:

- Implement rules to restrict read/write access based on user roles (e.g., only allow users to read/write their messages).

Data Encryption:

- Ensure data in transit is encrypted using HTTPS and utilize Firebase's built-in security features.

User Privacy:

- Provide users with options to control visibility (e.g., online status) and manage personal data.

5. Performance Optimization:

Scalability:

- Leverage Firebase's automatic scaling capabilities to handle growing user bases without performance degradation.

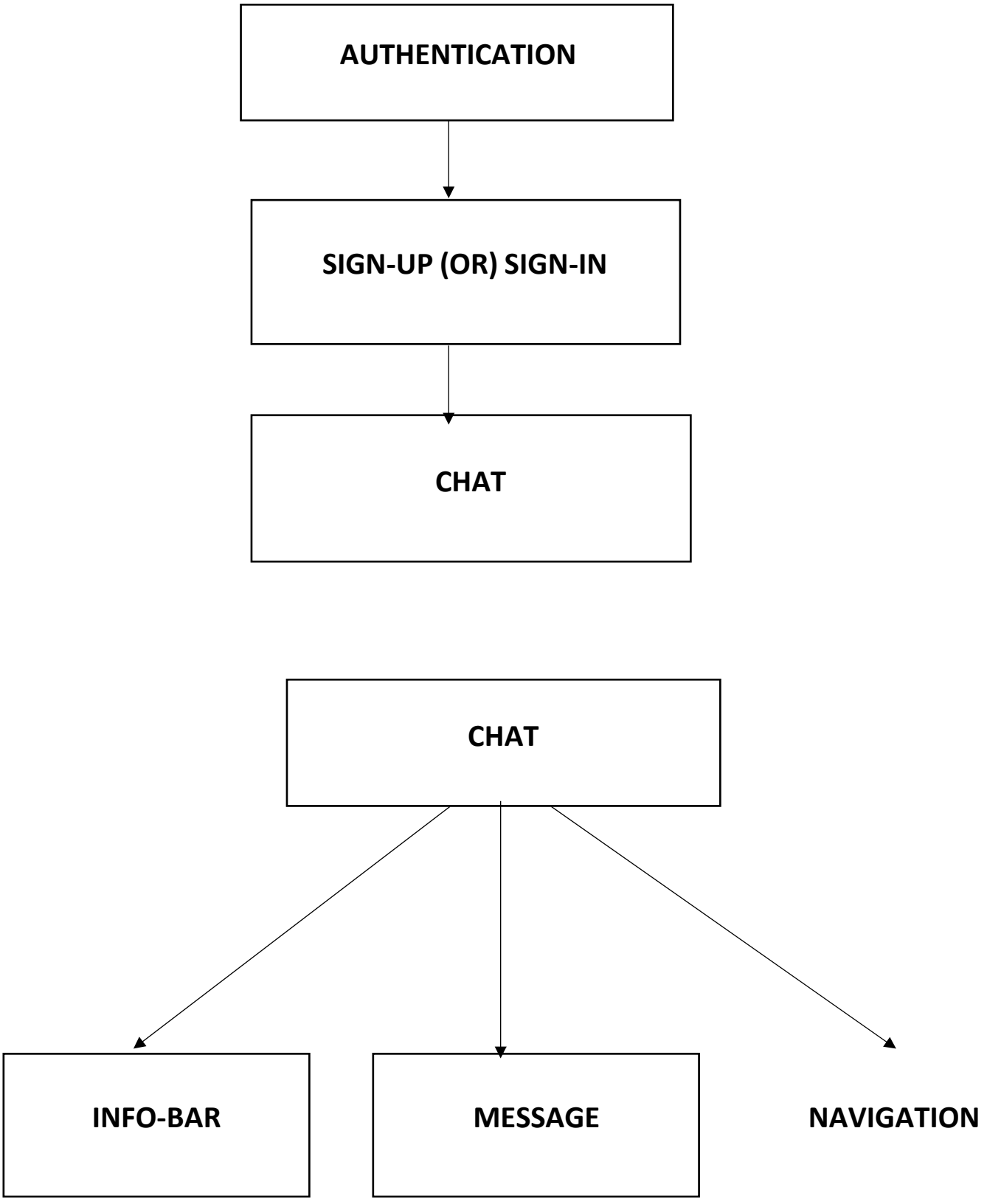
Caching:

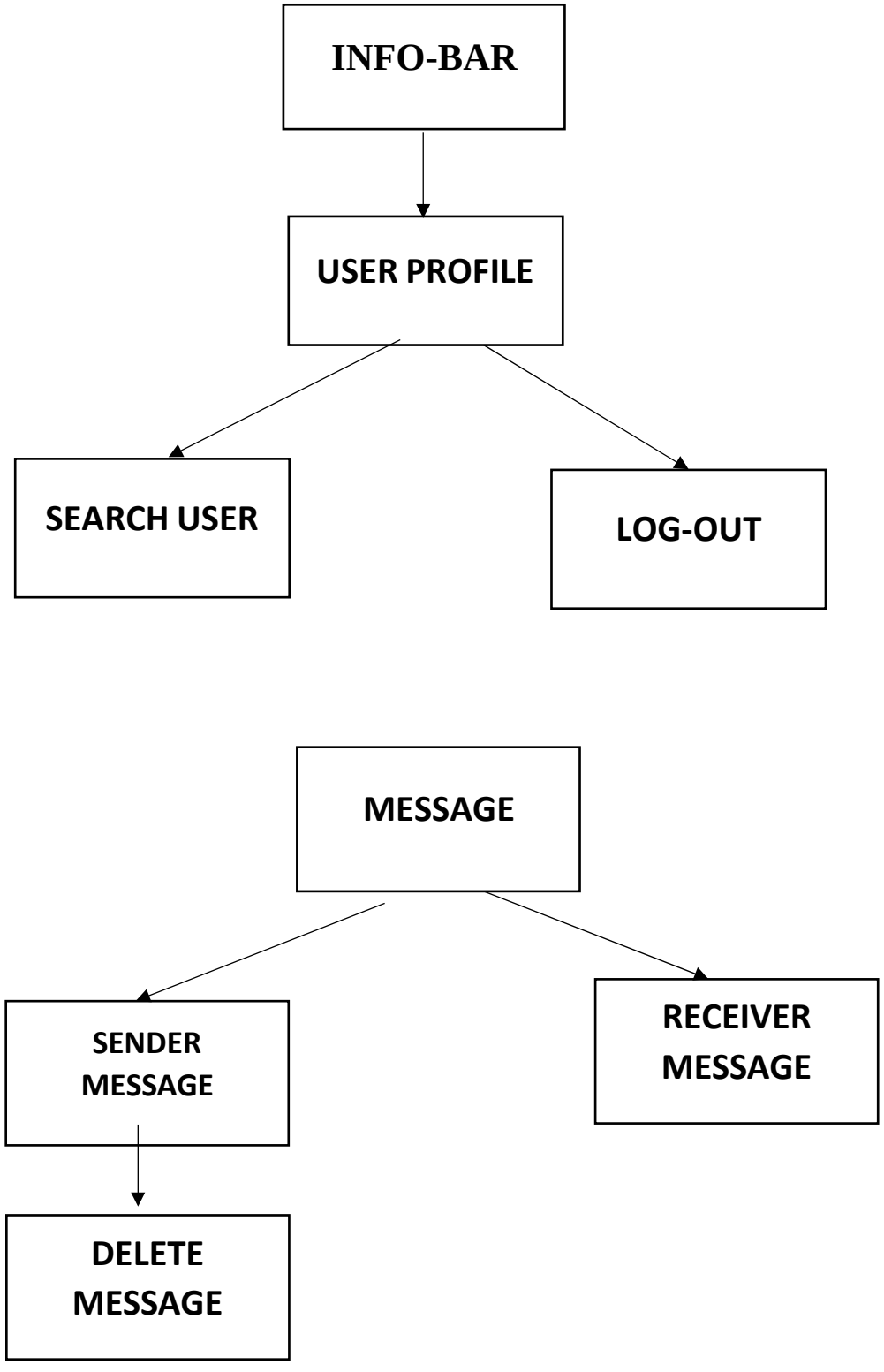
- Implement caching mechanisms (e.g., local storage) to improve loading times for frequently accessed data.

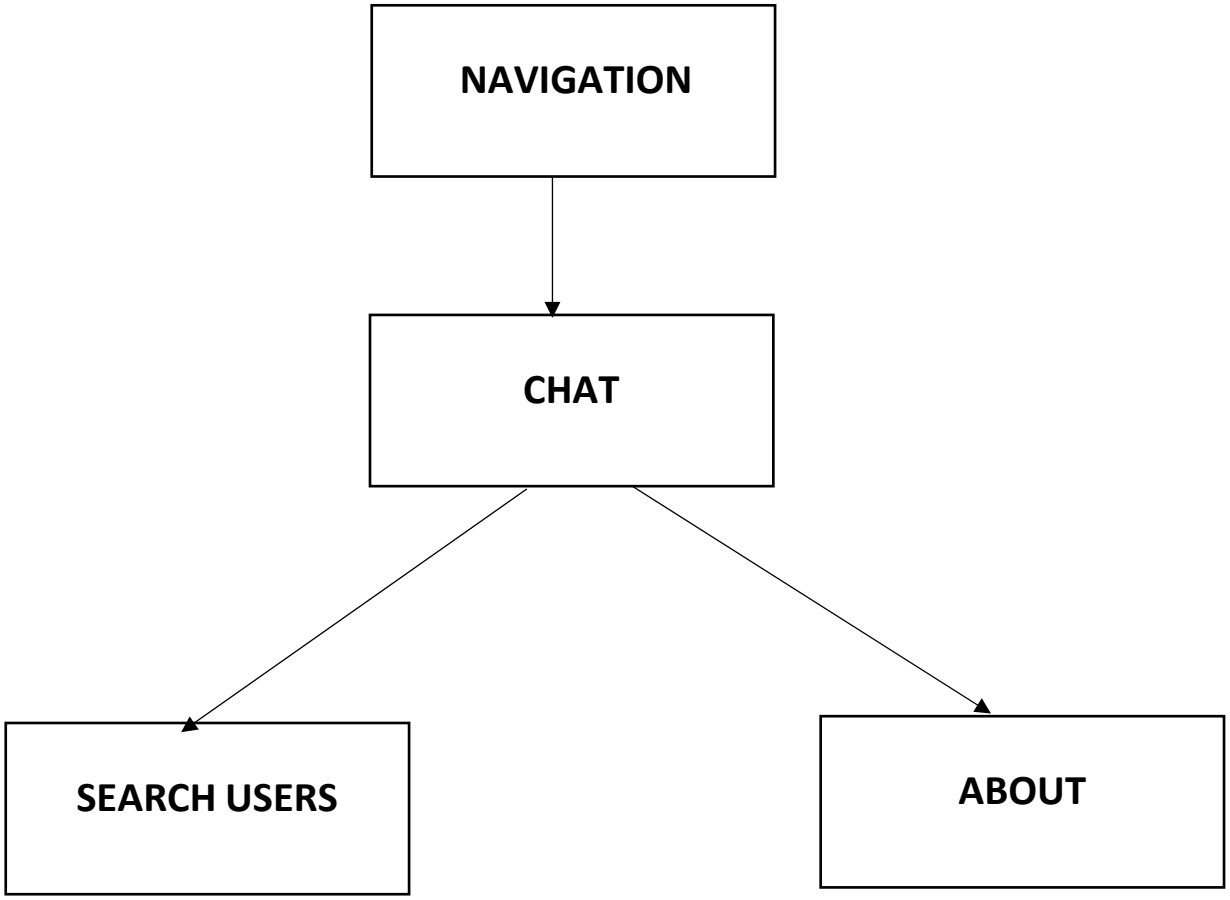
Load Testing:

- Regularly perform load testing to ensure the application can handle peak usage effectively.

4.2 DATA FLOW DIAGRAM




INFO-BAR
USER PROFILE
SEARCH USER
LOG-OUT
MESSAGE
SENDER MESSAGE
RECEIVER MESSAGE
DELETE MESSAGE


NAVIGATION
CHAT
SEARCH USERS
ABOUT

CHAPTER 5

SYSTEM IMPLEMENTATION

The implementation of the real-time chat application involves several key phases, each ensuring that the application functions as intended while delivering a seamless user experience. Below is a detailed breakdown of the implementation process, covering the front-end and back-end development, integration of features, and deployment.

1. Front-End Development:

Technologies Used:

- React.js: The core framework for building the user interface.
- Material-UI (MUI): A component library that provides pre-designed, customizable UI components for a modern look and feel.

Implementation Steps:

- Setting Up the Environment:
- Initialize a React application using Create React App.
- Install Material-UI and other necessary libraries (e.g., React Router for navigation, Redux for state management).

Component Development:

- Create reusable components such as:
- Login Component: Handles user authentication through Google Sign-In.
- Chat Interface: Displays the conversation window, input field, and message history.
- User List: Dynamically displays active users based on real-time updates.
- Group Chat Management: Interfaces for creating and managing group chats.

State Management:

- Utilize React Context or Redux to manage application state, including user authentication status, message history, and active user tracking.

Real-Time Updates:

- Implement Firebase Firestore listeners to fetch and display messages and user status updates in real-time.

2. Back-End Development:

Technologies Used:

- Firebase:Serves as the backend for authentication and data storage.

Implementation Steps:

- Firebase Setup:
- Create a Firebase project and enable Firestore, Realtime Database, and Authentication.

User Authentication:

- Integrate Google Authentication to facilitate user login.
- Configure Firebase Authentication settings to manage user sessions securely.

Database Design:

- Structure the Firestore database to include collections for users and messages:
- Users Collection: Stores user profiles and online status.
- Messages Collection: Stores message content, sender/receiver IDs, and timestamps.

Security Rules:

- Set up Firestore security rules to ensure users can only access their data, preventing unauthorized access.

3. Feature Integration:

- Key Features Implementation:
- Real-Time Communication:
- Use Firestore listeners to update the chat interface in real time as new messages are sent or received.

Dynamic User Tracking:

- Implement a feature that tracks active users and updates the user list dynamically, enabling users to see who is online.

Message History:

- Fetch and display a comprehensive message history for each conversation, allowing users to revisit past discussions easily.

4. Performance Optimization:

Optimization Strategies:

Data Retrieval:

- Utilize Firebase's capabilities for efficient data retrieval, ensuring minimal latency during message exchanges.

Caching:

- Implement caching strategies (e.g., local storage) to store frequently accessed data and improve loading times.

Load Testing:

- Perform load testing to evaluate application performance under various conditions and ensure scalability.

5. Deployment:

Deployment Steps:

- Frontend Deployment
- Use Netlify or Vercel to deploy the React application, ensuring a fast and reliable hosting solution.
- Backend Deployment
- Host the Firebase project and configure it for production, ensuring security and performance settings are properly adjusted.
- Domain and SSL
- Register a domain name and enable HTTPS for secure communication.

6. User Testing and Feedback:

- User Testing:
- Conduct user testing sessions to gather feedback on the application's usability and performance.
- Identify areas for improvement and iterate on the design and functionality based on user insights.

MODULE DESCRIPTIONS:

1. Authentication Module:

Description:

The Authentication module is a critical component of the real-time chat application, responsible for managing user login and registration processes. It utilizes Firebase Google Authentication to streamline user access while maintaining a high level of security. This module allows users to sign in using their Google accounts, providing a hassle-free experience by eliminating the need for traditional username and password combinations. Once authenticated, users gain access to the chat platform and its features, ensuring that sensitive data and conversations are protected from unauthorized access.

Purposes:

1. Secure Access:

The primary purpose of the authentication module is to ensure that only verified users can access the chat application. By requiring Google Authentication, it minimizes the risk of unauthorized access and protects user data.

2. User Convenience:

By allowing users to log in using their existing Google accounts, the module simplifies the login process. This not only enhances user experience but also reduces barriers to entry, encouraging more users to join and engage with the platform.

3. User Management:

The module tracks user sessions and manages user profiles, storing relevant information such as user IDs, email addresses, and profile pictures. This facilitates personalized experiences within the chat application.

4. Session Security:

It implements session management features to maintain secure user sessions. This includes automatic session expiration after periods of inactivity and secure handling of user credentials.

5. Data Protection:

By integrating secure authentication protocols, the module protects sensitive user information and chat data. It ensures that user credentials are not exposed or stored insecurely.

6. Scalability:

The module is designed to handle a growing number of users efficiently. As the user base expands, Firebase's authentication service can scale seamlessly, providing a robust solution for user management.

7. Integration with Other Features:

The authentication module is tightly integrated with other application features, such as real-time messaging and user tracking. This ensures that user identities are consistently verified across all functionalities, enhancing the overall coherence of the application.

2.Search Users Module:

Description:

The Search Users module is an essential component of the real-time chat application, enabling users to find and connect with other registered users quickly and efficiently. This module provides a user-friendly interface where individuals can input search queries—such as usernames or email addresses—to locate other users within the platform. The search functionality is designed to deliver instant results, allowing users to easily discover potential conversation partners or group members.

Purposes:

1. User Discovery:

The primary purpose of the Search Users module is to facilitate the discovery of other users within the application. This encourages social interaction and expands users' networks by allowing them to find friends, colleagues, or new contacts.

2. Efficient Communication Initiation:

By enabling users to search for others quickly, the module streamlines the process of initiating conversations. This fosters more spontaneous interactions and enhances user engagement within the chat platform.

3. Real-Time Results:

The module is designed for speed and efficiency, providing real-time search results as users type their queries. This responsiveness improves user experience and reduces frustration, allowing for a seamless exploration of user profiles.

4. Filter and Sort Options:

The module may include options to filter or sort search results based on criteria such as online status, mutual connections, or recent activity. This additional functionality enhances the search experience and helps users find the most relevant contacts.

5. User Privacy:

While facilitating user discovery, the module ensures that privacy settings are respected. Only users who have opted to be discoverable will appear in search results, protecting user privacy and preferences.

6. Integration with User Profiles:

The Search Users module is integrated with user profiles, allowing users to view detailed information about potential contacts (e.g., profile pictures, status messages) directly from the search results. This helps users make informed decisions about who to connect with.

7. Enhancing Community Engagement:

By simplifying the process of finding and connecting with others, this module contributes to a more vibrant community within the chat application. Increased interactions lead to stronger user engagement and a more active user base.

3. Private Chat (One-to-One) Module:

Description:

The Private Chat module facilitates one-to-one messaging between users within the real-time chat application. This module allows users to engage in private conversations, sharing text messages, images, and other media securely. Leveraging Firebase's onSnapshot listeners, the module ensures that all messages are updated in real time, providing a seamless communication experience. This means that any message sent or received is instantly reflected on all connected devices, allowing users to communicate without delays.

Purposes:

1. Real-Time Communication:

The primary purpose of the Private Chat module is to enable instant communication between users. By utilizing Firebase's onSnapshot listeners, messages are delivered and displayed in real-time, enhancing the immediacy of conversations.

2. User Privacy:

The module supports private interactions, ensuring that conversations are only visible to the participating users. This focus on privacy fosters a secure environment for sharing sensitive or personal information.

3. Message Synchronization:

It ensures that messages are synchronized across all devices. Whether users are on a mobile device or desktop, they can seamlessly transition between platforms without losing the context of their conversations.

4. User-Friendly Interface:

The Private Chat module is designed to be intuitive and easy to navigate. Features such as chat bubbles, timestamps, and read receipts enhance the user experience, making it simple for users to follow conversations.

5. Media Sharing:

In addition to text messaging, the module supports the sharing of images and other media types. This functionality enriches conversations and allows users to express themselves more fully.

6. Notifications:

The module can integrate notification features to alert users of new messages, ensuring they remain engaged and responsive to conversations even when not actively using the application.

7. Message History:

The module maintains a comprehensive message history for each private chat, allowing users to revisit past conversations. This feature is essential for maintaining context and continuity in discussions.

4. Group Chat Module:

Description:

The Group Chat module is a key feature of the real-time chat application, enabling multiple users to engage in conversations within the same chat room. This module allows users to create and join groups easily, facilitating collaborative communication for teams, friends, or communities. With real-time updates powered by Firebase, messages sent in group chats are instantly synchronized across all participants' devices, ensuring that everyone is on the same page. The interface typically includes features such as user mentions, notifications, and group settings to enhance the overall experience.

Purposes:

1. Collaborative Communication:

The primary purpose of the Group Chat module is to facilitate communication among multiple users simultaneously. It enables collaborative discussions, making it ideal for teams working on projects or groups planning events.

2. Real-Time Updates:

Utilizing Firebase's real-time capabilities, the module ensures that all messages are instantly visible to every participant in the group chat. This real-time communication fosters engagement and keeps conversations flowing without delays.

3. Dynamic Group Creation:

Users can easily create and manage groups based on their needs, whether for casual chats, study groups, or work-related discussions. This flexibility allows for quick adaptation to changing communication needs.

4. Enhanced User Interaction:

The module may include features like user mentions (tagging users in messages) and reaction options, allowing for more interactive and engaging conversations. These features help participants feel more involved in discussions.

5. Notification System:

To keep users informed of new messages, the module can incorporate notification features that alert participants when there are updates in the group chat. This ensures that users don't miss important messages, even when they are not actively viewing the chat.

6. Group Settings and Management:

Administrators can manage group settings, including adding or removing members, changing group names, and setting permissions. This control enhances the organization and structure of group interactions.

7. Message History:

The Group Chat module maintains a history of all messages exchanged within the group, allowing participants to reference past discussions. This feature is crucial for maintaining context and continuity in ongoing conversations.

5. Firebase onSnapshot Modules:

Description:

The Firebase onSnapshot modules are integral to the real-time chat application, providing real-time data synchronization across the platform. Utilizing Firebase's onSnapshot listeners, these modules listen for changes in the database—such as new messages, updates to user statuses, or alterations in chat data—and automatically reflect those changes in the application. This ensures that users experience a seamless and up-to-date interface, with real-time updates delivered without the need for manual refreshing or reloading.

Purposes:

1. **Real-Time Updates:**

The primary purpose of the onSnapshot modules is to deliver real-time updates to users. By immediately reflecting changes in messages and user statuses, these modules enhance the communication experience, allowing conversations to flow naturally and dynamically.

2. **Data Consistency:**

The modules ensure that all users have access to the most current data. This consistency is crucial for effective communication, as it minimizes confusion and ensures that everyone is viewing the same information at the same time.

3. **Efficient Resource Use:**

By leveraging Firebase's built-in real-time capabilities, the onSnapshot modules reduce the need for continuous polling or manual data refreshes. This optimizes resource use and improves overall application performance.

4. **User Engagement:**

Real-time updates keep users engaged and informed about ongoing conversations and activities. Notifications about new messages or changes in user status foster a more interactive and lively chat environment.

5. **Error Handling:**

The modules can also incorporate error handling mechanisms to manage potential issues during data synchronization. This ensures that users receive clear feedback in case of any problems, maintaining a reliable user experience.

6. **Scalability:**

As the user base grows, the onSnapshot modules can scale efficiently with Firebase's infrastructure. This capability ensures that real-time updates remain quick and reliable, regardless of the number of active users or messages being exchanged.

7. **Integration with Other Features:**

The onSnapshot modules work seamlessly with other application features, such as user authentication and messaging systems. This integration creates a cohesive environment where all aspects of the chat application function harmoniously.

6. User Status (Online/Offline Activity) Module:

Description:

The User Status module is a vital feature of the real-time chat application, responsible for tracking and displaying users' online and offline statuses. By leveraging Firebase's real-time database capabilities, this module updates user status indicators—such as a green dot for online and a grey dot for offline—next to each user's profile in the application. This visual representation provides immediate feedback on user availability, enhancing the overall communication experience.

Purposes:

1. Availability Awareness:

The primary purpose of the User Status module is to inform users of each other's availability for conversations. By clearly displaying online or offline status, it helps users decide the best times to initiate chats.

2. Enhanced Communication Efficiency:

Knowing whether someone is online encourages users to engage in conversations when their contacts are available, reducing delays and improving communication efficiency.

3. Real-Time Updates:

Utilizing Firebase's real-time synchronization, the module ensures that user statuses are updated instantly. Changes in status (e.g., a user going offline) are reflected immediately, maintaining an accurate representation of user availability.

4. Improved User Engagement:

By providing visible indicators of availability, the module enhances user engagement. Users are more likely to participate in conversations when they can see that their contacts are online and ready to chat.

5. User Interaction Context:

The online/offline status adds context to interactions, allowing users to manage their expectations regarding response times. This understanding can lead to a more patient and considerate communication environment.

6. Customizable Privacy Settings:

The module can include privacy settings, allowing users to control whether they want to be visible as online or offline. This feature gives users the option to maintain their privacy while using the chat platform.

7. Integration with Notifications:

The User Status module can work alongside notification features, alerting users when their contacts come online. This integration fosters timely interactions and encourages more active participation in group or one-to-one chats.

7. Recently Interacted Users Module:

Description:

The Recently Interacted Users module is a valuable feature of the real-time chat application that displays a list of users with whom a participant has recently engaged in conversations. This list is organized based on the last conversation timestamp, allowing users to quickly access their most recent chats without the need to search through their entire contact list. This module is designed to streamline the user experience by providing immediate access to frequently contacted individuals.

Purposes:

1. Quick Access to Conversations:

The primary purpose of the Recently Interacted Users module is to enable users to quickly find and initiate conversations with their most recently contacted friends or colleagues. This saves time and effort, making the chat experience more efficient.

2. Enhanced User Convenience:

By displaying a concise list of recent interactions, the module enhances user convenience. Users can easily resume discussions without navigating through multiple menus or searching for contacts.

3. Increased Engagement:

This module encourages users to engage more frequently with their contacts. The ease of accessing recent conversations may lead to more interactions and foster stronger relationships among users.

4. Contextual Awareness:

By highlighting recently interacted users, the module provides contextual awareness of communication patterns. Users can easily remember and follow up on previous discussions, enhancing continuity in conversations.

5. User-Friendly Interface:

The module is designed with an intuitive interface that presents recent interactions clearly and visually. This user-friendly design ensures that users can navigate their recent chats effortlessly.

6. Adaptability:

The Recently Interacted Users module can adapt to include additional features, such as indicating unread messages or adding quick action buttons (e.g., "Resume Chat" or "Video Call"). This adaptability further enhances user engagement.

7. Privacy Considerations:

Users can be given the option to manage their visibility in the recently interacted list, allowing them to control whether their recent interactions are displayed to others. This feature respects user privacy while maintaining convenience.

8. User Profile and Activity Log Module:

Description:

The User Profile and Activity Log module is a crucial component of the real-time chat application, designed to maintain and display essential user information. This module showcases user profiles, including status messages, profile pictures, and last active timestamps, providing a snapshot of each user's availability and current mood. Additionally, it includes a basic activity log that records significant events, such as login and logout times, as well as timestamps for recent chats. This comprehensive view enhances the context surrounding each user, fostering better interactions within the application.

Purposes:

1. User Contextualization:

The primary purpose of this module is to provide context about each user. By displaying status messages and last active times, users can quickly understand the availability and current sentiments of their contacts, facilitating more meaningful interactions.

2. Enhanced Communication:

With detailed profile information, users can engage in more relevant conversations. Knowing a contact's status (e.g., "Busy" or "Available") helps tailor communication approaches, improving overall chat effectiveness.

3. Activity Transparency:

The activity log records important user actions, such as when they logged in or out and their recent interactions. This transparency fosters trust among users, as they can see their contacts' activity patterns.

4. Improved User Engagement:

By providing insights into user activity and status, the module encourages more engagement. Users are more likely to reach out to contacts who appear active or to check in on those who may have been offline for a while.

5. Customization and Personalization:

The module allows users to personalize their profiles with custom status messages, enhancing self-expression. This personalization contributes to a richer user experience and strengthens community connections.

6. Ease of Access:

Users can easily access the profile and activity log from the main interface, making it convenient to review a contact’s information without navigating away from ongoing conversations.

7. Privacy Controls:

Users should have the ability to manage their visibility settings, controlling who can see their profile information and activity logs. This respects user privacy while maintaining an engaging environment.

CHAPTER 6

System Testing for Real-Time Chat Application:

System testing is a crucial phase in the development of the real-time chat application, ensuring that all components function correctly and meet the specified requirements. This process involves various types of testing, focusing on performance, security, usability, and real-time functionality. Below is an overview of the testing strategy for the application.

1. Functional Testing:

Purpose:

To verify that all features of the application work as intended.

Testing Components:

- User Authentication:
- Test Google Authentication for login and ensure that user credentials are securely handled.
- Verify that the login process is seamless and that users can recover passwords if necessary.

Real-Time Communication:

- Test sending and receiving messages in real-time across different devices.
- Verify that messages appear instantly in chat windows without refresh.

Active User Tracking:

- Check that the dynamic user list updates in real-time as users log in and out.
- Ensure users can see the correct online/offline status of other participants.

Message History:

- Validate that users can access and navigate their message history.
- Test the retrieval of old messages and ensure the timestamps are accurate.

2. Performance Testing:

Purpose:

To assess the application's responsiveness and stability under various conditions.

Testing Components:

Load Testing:

- Simulate multiple users (e.g., hundreds or thousands) sending messages simultaneously to measure the application's response time.
- Identify any bottlenecks that may affect performance as user numbers increase.

Stress Testing:

- Push the application beyond its limits to observe how it handles extreme conditions.
- Monitor how the system recovers from failures or crashes.

Latency Testing:

- Measure the time taken for messages to be sent and received to ensure minimal latency.
- Test performance on different network conditions (e.g., 3G, 4G, Wi-Fi).

3. Security Testing:

Purpose:

To identify vulnerabilities in the application and ensure data protection.

Testing Components:

Authentication Security:

- Verify that user data is encrypted during transmission and stored securely.
- Test the implementation of Firestore security rules to restrict unauthorized access.

Data Privacy:

- Ensure that sensitive user information (e.g., email addresses) is not exposed in any way.
- Test the application's response to unauthorized access attempts.

Session Management:

- Validate that user sessions expire after a designated period of inactivity.
- Test for vulnerabilities such as session hijacking.

4. Usability Testing:

Purpose:

To assess the user experience and interface design.

Testing Components:

User Interface:

- Gather feedback from users regarding the intuitiveness of the UI, navigation, and overall design.
- Test the responsiveness of the application across different devices (mobile, tablet, desktop).

User Accessibility:

- Ensure the application is usable for individuals with varying levels of technical expertise.
- Verify compliance with accessibility standards (e.g., screen reader compatibility).

5. Regression Testing:

Purpose:

To ensure that new updates or bug fixes do not adversely affect existing functionality.

Testing Components:

- After implementing changes or enhancements, re-test all critical features of the application.
- Maintain a suite of test cases that can be run regularly to catch any issues early in the development cycle.

CHAPTER 7

CONCLUSION:

In conclusion, this real-time chat application stands out as a transformative solution for online communication, combining an exceptional user experience with stringent security measures and scalable architecture. By leveraging modern technologies such as React and Firebase, and incorporating user-friendly elements from Material-UI, the application is designed to cater to a diverse audience, ensuring ease of use for both tech-savvy individuals and those less familiar with digital tools. The integration of secure authentication processes, real-time user tracking, and efficient data management fosters a trustworthy and engaging environment for users. With features that support seamless communication and maintain context through message history, the application is well-equipped to meet a variety of personal and professional communication needs. Overall, it redefines the standards of online interaction, making it an invaluable platform for anyone seeking reliable and dynamic ways to connect with others.

CHAPTER 8

8.1 SOURCE CODE

CHAT.JS:

```
import React, { useContext, useEffect } from 'react'
import Message from './Message'
import Input from './Input'
import InfoBar from './InfoBar'
import { UserContext } from '../context/UserContext'
import FormDialog from '../component/MUI/FormDialog'
import FormDialogGrp from '../component/MUI/FormDialogGrp'
import TabBar from './TabBar'
import { ChatContext } from '../context/ChatContext'
import UserPage from '../pages/UserPage'
import GroupChatPage from '../pages/GroupChatPage'
import Users from '../pages/Users'
import ShowActiveUsers from './MUI/ShowActiveUsers'
const Chat = () => {
   const { user, setOpen, chatWithWho } = useContext(UserContext);
   const { chatPhase, setChatPhase } = useContext(ChatContext);
   useEffect(() => {
      if (!user?.displayName) setOpen(true);
   }, [user.displayName, setOpen, chatWithWho])
   return (
      <div className="" >
         <FormDialog />
         <FormDialogGrp />
         <div className="chat-app-container">
            {/* Left Side - Search Box and Search Items */}
```

```
        {/* Right Side - Users Page and Group Chat Users Page */}
        <div className="right-side">
          <div className="users-page">
            <UserPage />
            {/* Add more users */}
          </div>
          <div className="group-chat-users-page">
            <GroupChatPage />
            {/* Add more group chat users */}
          </div>
        </div>
        {/* Center Area */}
        <div className="center-area">
          {/* Top Info Bar */}
          <div className="info-bar">
            <InfoBar />
          </div>
          {/* Message Display Area */}
          <div className="message-display">
            { chatPhase === 'user' && <ShowActiveUsers />}
            {!chatWithWho.length > 0 && chatPhase === 'messages' && <Message />}
            {/* Add more messages */}
          </div>
          {/* Bottom Input Box */}
          <div className="input-box">
            <Input />
          </div>
        </div>
      </div>
    </div>
```

```
    )
}
```

INFOBAR.JS:

```
import React, { useContext } from 'react';
import { UserContext } from '../context/UserContext';
import Logout from '../component/auth/Logout';
import UsersAvatars from './MUI/UsersAvatars';
const InfoBar = () => {
  const { user, setOpen, chatWithWho } = useContext(UserContext);
  const openDialog = () => {
    setOpen(true);
  }
  return (
    <div className='infoBar' style={{ position: 'relative' }}>
      <div >
        {chatWithWho.length === 0 ? (
          <div className='profile' onClick={openDialog} >
            <img src={user?.photoURL} alt="" style={photoURL} />
            <b>{user?.displayName}</b>
          </div>
        ) : (
          <UsersAvatars chatWithWho={chatWithWho} />
        )
        }
      </div>
      {chatWithWho.length !== 0 &&
        <div style={{ marginTop: '10px' }}>
          <></>
        </div>}
      <div className='logout-container'>
```

```
        <Logout />
      </div>
    </div>
  )
}
const photoURL = { width: '50px', height: '50px', borderRadius: '50%' };
```

INPUT.JS:

```
import React, { useContext } from 'react';
import { doc, setDoc, updateDoc, getDoc, arrayUnion, Timestamp } from 'firebase/firestore';
import { UserContext } from '../context/UserContext';
import { ChatContext } from '../context/ChatContext';
import { db } from '../config/firebase-config';
const Input = () => {
    const { user, chatWithWho } = useContext(UserContext);
    const { message, setMessage } = useContext(ChatContext);
    const getUniqueChatId = (user, chatWithWho) => {
        if (Array.isArray(chatWithWho.members)) {
            // If chatWithWho is a group, return the group ID
            return chatWithWho.groupId;
        } else {
            // For one-to-one chats, sort user UIDs
            const sortedUids = [user.uid, chatWithWho.uid].sort();
            return sortedUids.join('');
        }
    };
    const sendMessage = async (e) => {
        e.preventDefault();
        if (message.trim()) {
            try {
                const receivedMessage = {
```

```
            id: new Date().getTime(),
            name: user?.displayName || 'Anonymous',
            img: user?.photoURL,
            time: Timestamp.now(),
            text: message,
            sender: user?.uid
        };
        const chatId = getUniqueChatId(user, chatWithWho);
        const chatRef = doc(db, 'chats', chatId);
        const chatDoc = await getDoc(chatRef);
        if (chatDoc.exists()) {
            await updateDoc(chatRef, {
                messages: arrayUnion(receivedMessage) // Use arrayUnion to avoid
duplicates
            });
        } else {
            await setDoc(chatRef, {
                messages: [receivedMessage]
            });
        }
        setMessage('');
        // Update the userPage collection with the latest message
        const userPageRef = doc(db, 'userPage', chatWithWho?.uid);
        const userPageDoc = await getDoc(userPageRef);
        const newChatData = {
            id: user?.uid,
            name: user?.displayName,
            img: user?.photoURL,
            text: message,
            time: Timestamp.now(),
        };
```

```
            if (userPageDoc.exists()) {
                const currentData = userPageDoc.data().chats || [];
                const updatedData = currentData.filter(chat => chat.id !== newChatData.id);
                updatedData.push(newChatData);
                await updateDoc(userPageRef, {
                    chats: updatedData
                });
            } else {
                await setDoc(userPageRef, { chats: [newChatData] });
            }
        } catch (error) {
            console.error('Error sending message: ', error);
        }
    }
};
if (chatWithWho.length === 0 ) return false;
return (
    <div className='input-box'>
        <form className="form">
            <input
                className="input"
                type="text"
                placeholder="Type here..."
                value={message}
                onChange={({ target: { value } }) => setMessage(value)}
                onKeyPress={event => event.key === 'Enter' ? sendMessage(event) : null}
            />
            <button className="sendButton" onClick={e =>
sendMessage(e)}>Send</button>
        </form>
```

```
        </div>
    );
};
```

MESSAGE.JS:

```
import React, { useContext, useEffect, useState } from 'react';
import ScrollToBottom from 'react-scroll-to-bottom';
import { ChatContext } from '../context/ChatContext';
import { UserContext } from '../context/UserContext';
import { doc, onSnapshot, updateDoc, arrayRemove } from 'firebase/firestore';
import { db } from '../config/firebase-config';
import UserPage from '../pages/UserPage';
import { CircularProgress } from '@mui/material';
import { convertTimestamp } from '../utils/commonFunctions';
import GroupChatPage from '../pages/GroupChatPage';
const Message = () => {
    const [openMessageIndex, setOpenMessageIndex] = useState(null);
    const [loading, setLoading] = useState(true);
    const { messages, setMessages } = useContext(ChatContext);
    const { user, chatWithWho } = useContext(UserContext);
    const getUniqueChatId = (user, chatWithWho) => {
        if (Array.isArray(chatWithWho.members)) {
            // If chatWithWho is a group, return the group ID
            return chatWithWho.groupId;
        } else {
            // For one-to-one chats, sort user UIDs
            const sortedUids = [user.uid, chatWithWho.uid].sort();
            return sortedUids.join('');
        }
    };
```

```
useEffect(() => {
    if (user && chatWithWho) {
        setLoading(true); // Start loading when switching chats

        const chatId = getUniqueChatId(user, chatWithWho);
        const chatRef = doc(db, "chats", chatId);
        const unsubscribe = onSnapshot(chatRef, (snapshot) => {
            if (snapshot.exists()) {
                const data = snapshot.data();
                setMessages(data.messages || []);
            } else {
                setMessages([]); // Reset messages if no data
            }
            setLoading(false); // Stop loading once data is fetched
        });
        return () => unsubscribe(); // Unsubscribe from the previous chat when switching
    }
}, [user, chatWithWho, setMessages]);
const deleteChat = async (msg) => {
    try {
        const chatId = getUniqueChatId(user, chatWithWho);
        const chatRef = doc(db, "chats", chatId);
        await updateDoc(chatRef, {
            messages: arrayRemove(msg)
        });
    } catch (error) {
        console.error("Error deleting message: ", error);
    }
```

```
  };
  const toggleDeleteIcon = (index) => {
    setOpenMessageIndex(prevIndex => prevIndex === index ? null : index);
  }
  if (loading) return (
    <div className='loading-container'>
      <CircularProgress />
    </div>
  );
  return (
    <div className={`${chatWithWho?.length === 0 ? 'message-container' : "message-container-bg"}`} >
      {/* <ThemeDialog /> */}
      <ScrollToBottom checkInterval={100} className={`${chatWithWho.length !== 0 && "message-container-scroll"}`}>
        {messages?.map((msg, index) => (
          <div
            key={index}
            className={`message ${msg.sender === user.uid ? 'sent' : 'received'}`}
            onClick={() => msg.sender === user.uid && toggleDeleteIcon(index)}
          >
            <div className='message-content'>
              <p className='message-img'>
                <img src={msg?.img} alt="" style={photoURL} />
                <p className='message-name'>{msg.name}</p>
                {msg.sender === user.uid && openMessageIndex === index && (
                  <b style={{ flex: 1, cursor: 'pointer' }} onClick={() => deleteChat(msg)}>∅</b>
                )}
              </p>
              <p className='message-text'>{msg?.text}</p>
```

```
                    <p className='message-time'>{convertTimestamp(msg?.time)}</p>
                  </div>
                </div>
              ))}
          </ScrollToBottom>
        </div>
    );
};
const photoURL = { width: '20px', height: '20px', borderRadius: '50%' };
```

TABAR.JS:

```
import React, { useContext } from 'react'
import Diversity3SharpIcon from '@mui/icons-material/Diversity3Sharp';
import PersonSharpIcon from '@mui/icons-material/PersonSharp';
import { ChatContext } from '../context/ChatContext';
const TabBar = () => {
  const { setChatPhase } = useContext(ChatContext)
    const handleUser = ()=>{
        setChatPhase('user');
    }
    const handleGrp = ()=>{
        setChatPhase('group');
    }
  return (
    <div className='tab-bar' style={{ display: 'flex', justifyContent: 'space-around',
alignItems: 'center'}}>
        <PersonSharpIcon onClick={handleUser}/>
        <Diversity3SharpIcon onClick={handleGrp}/>
    </div>
  )
}
```

APP.CSS:

```
code {
  font-family: source-code-pro, Menlo, Monaco, Consolas, 'Courier New',
    monospace;
}
/* General App styles */
.App {
  display: flex;
  text-align: center;
  justify-content: center;
  place-items: center;
}
/* Hide scrollbar in WebKit browsers (Chrome, Safari, etc.) */
body {
  margin: 0;
  font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', 'Roboto', 'Oxygen',
    'Ubuntu', 'Cantarell', 'Fira Sans', 'Droid Sans', 'Helvetica Neue',
    sans-serif;
  -webkit-font-smoothing: antialiased;
  -moz-osx-font-smoothing: grayscale;
  /* background: url('./assets/svg/chat-bg.svg') no-repeat center center fixed; */
  background-size: cover;
  /* Ensures the background covers the entire body */
}
/* TabBar.css */
.tab-bar {
  display: flex;              /* Use flexbox for layout */
  justify-content: space-around; /* Space items evenly */
  align-items: center;        /* Center items vertically */
  background-color: #f9f9f9;  /* Optional: background color */
```

```
  padding: 10px;              /* Optional: padding for the bar */
  border-top: 1px solid #e0e0e0; /* Optional: top border */
  box-shadow: 0 -1px 5px rgba(0, 0, 0, 0.1); /* Optional: shadow for depth */
  height: 60px;               /* Optional: fixed height */
}
/* Optional: Add hover effect for icons */
.tab-bar svg:hover {
  cursor: pointer;            /* Change cursor to pointer on hover */
  transform: scale(1.1);      /* Scale up icon slightly on hover */
  transition: transform 0.2s; /* Smooth transition for scaling */
}
/* ChatComponent.css */
/* Container */
.chat-container {
  border-top: none;
  border: 1px solid #ddd;
  overflow-y: auto; /* Enable vertical scrolling */
  height: 250px; /* Set a fixed height */
  width: 100%; /* Set width to fill the container */
}
/* Hide scrollbar but keep scrolling enabled */
.chat-container::-webkit-scrollbar {
  display: none; /* Hide scrollbar for Chrome, Safari, and Opera */

.chat-container {
  scrollbar-width: none; /* Hide scrollbar for Firefox */
  -ms-overflow-style: none; /* Hide scrollbar for Internet Explorer and Edge */
}
```

```
/* Chat container styling */
.chat-container {
  width: 570px;
}
/* loading */
.loading-container {
  display: flex;
  justify-content: center;
  align-items: center;
  height: 100vh;
  /* Adjust based on your needs */
  width: 100%;
}
/* sign-in center */
.Auth {
  margin-top: 50px;
}
/* chatWithWho */
.chatWithWho {
  display: flex;
  flex-direction: row;
  justify-content: center;
}
.chatWithWho b {
  color: #000000;
  /* Optional: Change the color to grey for better differentiation */
}
/* logout */
.logout-container {
  display: flex;
```

```
  justify-content: flex-end;
}
.log-out-btn {
  align-self: center;
  border: none;
  outline: none;
  background-color: #F5F5F5;
  cursor: pointer;
}
/* message-container */
.message-container {
  position: relative;
  flex: 1;
  overflow-y: auto;
  padding: 8px;
  background-color: #f9f9f9;
  border-bottom: 1px solid #ddd;
  display: flex;
  flex-direction: column;
}
.message-container-scroll {
  position: relative;
  flex: 1;
  overflow-y: auto;
  padding: 8px;
  display: flex;
  flex-direction: column;
}
.message-container-bg {
  background-size: cover;
```

```
  background-position: center;
  background-repeat: no-repeat;
  position: relative;
  flex: 1;
  overflow-y: auto;
  padding: 8px;
  border-left: 2px solid #F5F5F5;
  border-right: 2px solid #F5F5F5;
  display: flex;
  flex-direction: column;
}
/* message styles */
.message {
  position: relative;
  margin-bottom: 8px;
  border: 1px solid #ddd;
  box-shadow: 0 2px 3px rgba(0, 0, 0, 0.1);
  display: flex;
  flex-direction: column;
  max-width: 40%;
  word-break: break-word;
  font-size: 0.85em;
  line-height: 1.2;
}
/* sent messages */
.message.sent {
  border-top-right-radius: 32px;
  border-bottom-left-radius: 32px;
  border-bottom-right-radius: 32px;
  align-self: flex-start;
```

```
  background-color: #e1f5fe;
  border-color: #b3e5fc;
}
/* received messages */
.message.received {
  border-top-left-radius: 32px;
  border-bottom-left-radius: 32px;
  border-top-right-radius: 32px;
  overflow: hidden;
  margin-left: auto;
  /* Push received messages to the right */
  background-color: #ffffff;
  border-color: #cfd8dc;
}
.message.received .message-name {
  margin: 0 0 4px 0;
  color: #333;
  padding: 0;
  /* Ensure no padding is causing overflow issues */
  box-sizing: border-box;
  align-self: flex-start;
  margin-left: 10px;
}
.message.received .message-text {
  font-weight: bold;
  font-size: large;
  margin: 0 0 4px 0;
  color: #444;
  margin-bottom: 10px;
```

```
.message.received .message-time {
  font-size: 0.7em;
  color: #777;
  margin: 0;
  text-align: left;
  margin-left: 20px;
}
.message.received .message-img {
  display: flex;
  flex-direction: row-reverse;
  margin-right: 10px;
}
/* sender name */
.message-name {
  margin: 0 0 4px 0;
  color: #333;
  padding: 0;
  /* Ensure no padding is causing overflow issues */
  box-sizing: border-box;
  align-self: flex-start;
  margin-left: 10px;
}
/* message text */
.message-text {
  font-weight: bold;
  font-size: large;
  margin: 0 0 4px 0;
  color: #444;
  margin-bottom: 10px;
}
```

```
/* message time */
.message-time {
  font-size: 0.7em;
  color: #777;
  margin: 0;
  text-align: right;
  margin-right: 20px;
}
/* message-img */
.message-img {
  display: flex;
  flex-direction: row;
  margin-left: 10px;
}
/* styling for message bubbles */
.message.sent::before {
  content: "";
  position: absolute;
  top: 10px;
  left: -10px;
  width: 0;
  height: 0;
  border-width: 10px;
  border-style: solid;
  border-color: transparent #e1f5fe transparent transparent;
}
.message.received::before {
  content: "";
  position: absolute;
```

```
  top: 10px;
  right: -10px;
  width: 0;
  height: 0;
  border-width: 10px;
  border-style: solid;
  border-color: transparent transparent transparent #ffffff;
}
/* Messages display area */
.messages {
  flex: 1;
  /* Take up remaining space */
  overflow-y: auto;
  /* Allow scrolling */
  padding: 10px;
  background-color: #fff;
  border-bottom: 1px solid #ddd;
}
/* Input form styling */
.input-box {
  border: 1px solid #ddd;
}
.form {
  display: flex;
  flex-direction: row;
  padding: 10px;
  align-items: center;
}
.form .input {
  flex: 1;
```

```
  padding: 10px;
  border: none;
  border-radius: 5px;
  margin-right: 10px;
  font-size: 1em;
}
.form .input:focus {
  border: none;
  /* No border on focus */
  outline: none;
  /* Remove default browser outline */
  box-shadow: none;
  /* Remove any default box-shadow that might appear */
}
.form .sendButton {
  background-color: #007bff;
  color: white;
  border: none;
  padding: 10px;
  border-radius: 5px;
  cursor: pointer;
  font-size: 1em;
}
.form .sendButton:hover {
  background-color: #0056b3;
}
/* infoBar */
/* InfoBar container styling */
.infoBar {
  display: flex;
```

```
  justify-content: space-between;
  align-items: center;
  background-color: #f5f5f5;
  padding: 10px;
  border-bottom: 1px solid #ddd;
  box-shadow: 0 1px 2px rgba(0, 0, 0, 0.1);
  position: relative;
}
/* Style for the user name */
.infoBar b {
  margin-left: 10px;
  font-size: 1.2em;
  color: #333;
}
.infoBar .profile {
  cursor: pointer;
  display: flex;
  flex-direction: row;
  align-items: center;
}
/* Additional styles if needed */
.infoBar img {
  object-fit: cover;
}
/* users */
.users-list {
  display: flex;
  flex-direction: column;
}
/* sign-ip and login and logout*/
```

```
.sign-Up,
.login {
  background-color: #fff;
  padding: 2rem;
  border-radius: 8px;
  box-shadow: 0 2px 10px rgba(0, 0, 0, 0.1);
  max-width: 400px;
  width: 100%;
  text-align: center;
  margin-left: auto;
  margin-right: auto;

.sign-Up-heading {
  margin-bottom: 1.5rem;
  color: #333;
}
.sign-Up-input {
  width: 100%;
  padding: 0.75rem;
  margin: 0.5rem 0;
  border: 1px solid #ccc;
  border-radius: 4px;
  font-size: 1rem;
}
.sign-Up-button {
  display: flex;
  align-items: center;
  justify-content: center;
  width: 100px;
```

```
  /* Adjust width as needed */
  height: 40px;
  /* Adjust height as needed */
  background-color: #007bff;
  /* Button background color */
  color: white;
  border: none;
  border-radius: 5px;
  cursor: pointer;
  position: relative;
}
.sign-Up-button:hover {
  background-color: #0056b3;
}
.login-heading {
  margin-bottom: 1.5rem;
  color: #333;
}
.login-input {
  width: 100%;
  padding: 0.75rem;
  margin: 0.5rem 0;
  border: 1px solid #ccc;
  border-radius: 4px;
  font-size: 1rem;
}
.login-button {
  display: flex;
  align-items: center;
  justify-content: center;
```

```
  width: 100px;
  /* Adjust width as needed */
  height: 40px;
  /* Adjust height as needed */
  background-color: #007bff;
  /* Button background color */
  color: white;
  border: none;
  border-radius: 5px;
  cursor: pointer;
  position: relative;
}
.login-button:hover {
  background-color: #0056b3;
}
.link-button {
  width: auto;
  padding: 0;
  background: none;
  color: #007bff;
  border: none;
  cursor: pointer;
  text-decoration: underline;
  font-size: 1rem;
  margin-top: 1rem;
}
.link-button:hover {
  text-decoration: none;
}
```

```
/* google login*/

.login-with-google-btn {

  transition: background-color .3s, box-shadow .3s;

  padding: 12px 16px 12px 42px;

  border: none;

  border-radius: 3px;

  box-shadow: 0 -1px 0 rgba(0, 0, 0, .04), 0 1px 1px rgba(0, 0, 0, .25);

  color: #757575;

  font-size: 14px;

  font-weight: 500;

  font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, Oxygen, Ubuntu,
Cantarell, "Fira Sans", "Droid Sans", "Helvetica Neue", sans-serif;

  background-image:
url(
SJodHRwOi8vd3d3LnczLm9yZy8yMDAwL3N2ZyI+PGcgZmlsbD0ibm9uZSIgZmlsbC1yd
WxlPSJldmVub2RkIj48cGF0aCBkPSJNMTcuNiA5LjJsLS4xLTEuOEg5djMuNGg0LjhDM
TMuNiAxMiAxMyAxMyAxMiAxMy42djIuMmgzYTguOCA4LjggMCAwIDAgMi42LTYuNnoiIGZpbGw9IiM0Mjg1RjQiIGZpbGwtcnVsZT0ibm9uemVybyIvPjxwYXRoIGQ9Ik05ID
E4YzIuNCAwIDQuNS0uOCA2LTIuMmwtMy0yLjJhNS40IDUuNCAwIDAgMS04LTIuOU
gxVjEzYTkgOSAwIDAgMCA4IDV6IiBmaWxsPSIjMzRBODUzIiBmaWxsLXJ1bGU9Im5
vbnplcm8iLz48cGF0aCBkPSJNNCAxMC43YTUuNCA1LjQgMCAwIDEgMC0zLjRWNUg
xYTkgOSAwIDAgMCAwIDhsMy0yLjN6IiBmaWxsPSIjRkJCQzA1IiBmaWxsLXJ1bGU9I
m5vbnplcm8iLz48cGF0aCBkPSJNOSAzLjZjMS4zIDAgMi41LjQgMy40IDEuM0wxNSAy
LjNBOSA5IDAgMCAwIDEgNWwzIDIuNGE1LjQgNS40IDAgMCAxIDUtMy43eiIgZmlsb
D0iI0VBNDMzNSIgZmlsbC1ydWxlPSJub256ZXJvIi8+PHBhdGggZD0iTTAgMGgxOHYx
OEgweiIvPjwvZz48L3N2Zz4=);

  background-color: white;

  background-repeat: no-repeat;

  background-position: 12px 11px;

  &:hover {

    box-shadow: 0 -1px 0 rgba(0, 0, 0, .04), 0 2px 4px rgba(0, 0, 0, .25);

  }

  &:active {

    background-color: #eeeeee;

  }
```

```
  &:focus {
    outline: none;
    box-shadow:
      0 -1px 0 rgba(0, 0, 0, .04),
      0 2px 4px rgba(0, 0, 0, .25),
      0 0 0 3px #c8dafc;
  }
  &:disabled {
    filter: grayscale(100%);
    background-color: #ebebeb;
    box-shadow: 0 -1px 0 rgba(0, 0, 0, .04), 0 1px 1px rgba(0, 0, 0, .25);
    cursor: not-allowed;
  }
}
.logout-button {
  background-color: #e0f7fa; /* Light blue background */
  border-radius: 8px;
  border:  none;
  cursor: pointer;
  padding: 10px 15px;
  display: flex;
  align-items: center;
  justify-content: center;
  gap: 10px;
  transition: box-shadow 0.3s ease;
}
.logout-button:hover {
  box-shadow: 0 0 15px 3px rgba(30, 144, 255, 0.8); /* Blue glow effect */
}
```

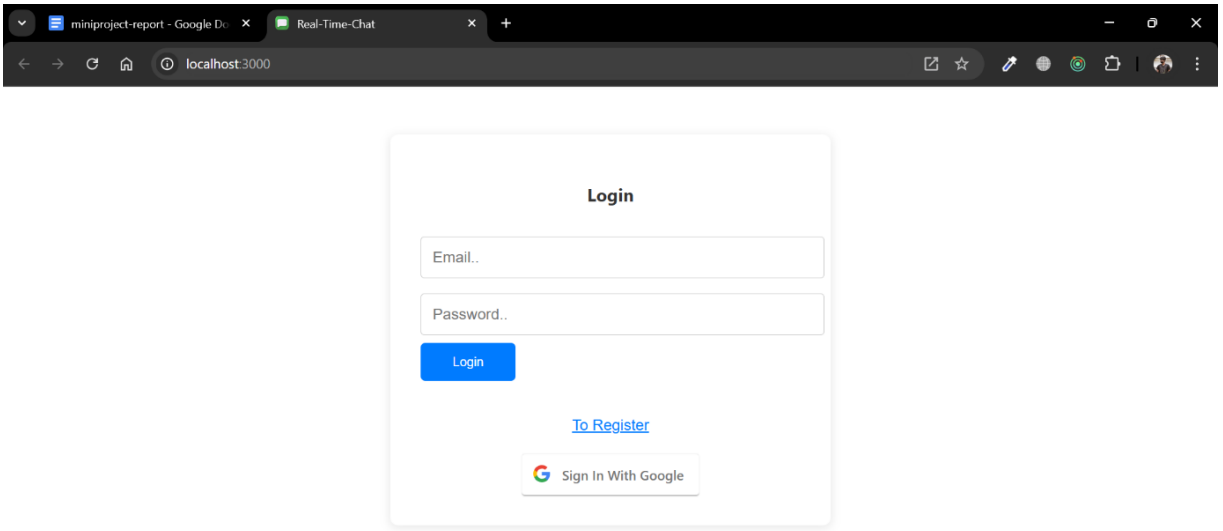
```
.logout-button:disabled {
  cursor: not-allowed;
}
.logout-avatar {
  width: 35px;
  height: 35px;
.react-loading {
  margin-right: 10px;
}
/* layout */
.chat-app-container {
  display: grid;
  grid-template-columns: 1fr 3fr; /* Sidebar, main chat area, sidebar */
  height: 100vh; /* Full viewport height */
  gap: 5px; /* Gap between columns */
}
/* Left Side */
.left-side {
  display: flex;
  flex-direction: column;
  border-right: 1px solid #ddd;
  padding: 10px;
  overflow-y: auto; /* Enable scrolling only in the left side if needed */
}
.search-box input {
  width: 100%;
  padding: 8px;
  border: 1px solid #ccc;
  border-radius: 4px;
```

```
}
.search-items {
  margin-top: 10px;
}
/* Center Area */
.center-area {
  display: flex;
  flex-direction: column;
  border-left: 1px solid #ddd;
  border-right: 1px solid #ddd;
  height: calc(100vh); /* Adjust height to accommodate padding */
}
.info-bar {
  background-color: #f1f1f1;
  border-bottom: 1px solid #ddd;
}
.message-display {
  flex: 1; /* Allows the message display to take up remaining space */
  overflow-y: auto; /* Enable vertical scrolling for messages if needed */
}
.users-page {
  margin-bottom: 10px;
  border-bottom: 1px solid #ddd;
  padding-bottom: 10px;
}
.group-chat-users-page {
  margin-top: -10px;
}
.users-page {
  margin-top: -10px;
```

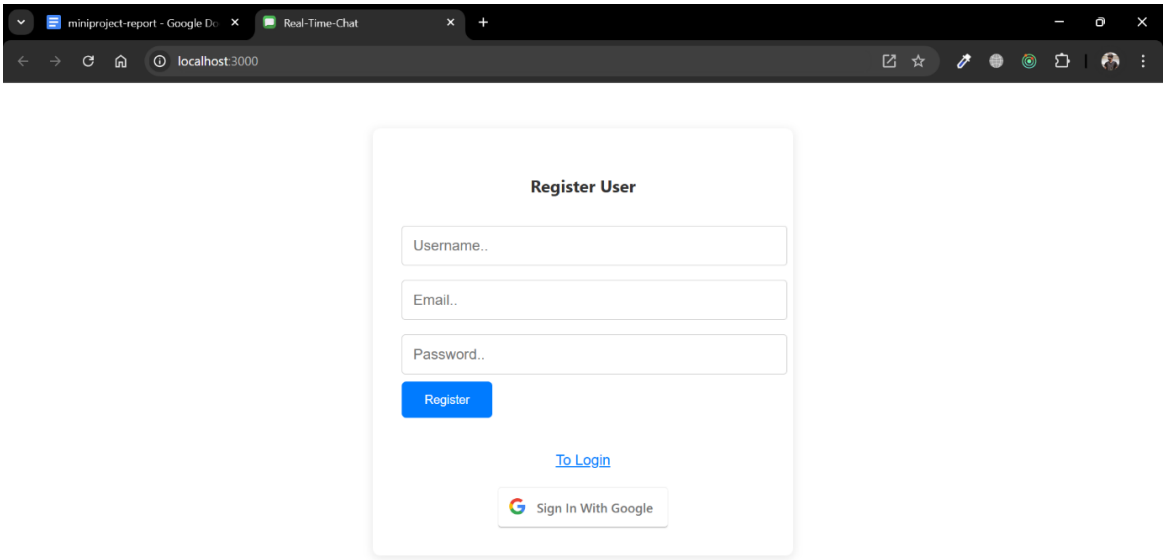
```
}
/* Ensure that the layout does not overflow */
html, body {
  height: 100%;
  margin: 0; /* Remove default margin */
  overflow: hidden; /* Prevent overall scrolling */
}
.scrollable {
  width: 300px;
  height: 200px;
  overflow: auto; /* Enables scrolling */
  border: 1px solid #ccc; /* Optional styling */
}
/* Style for WebKit browsers */
.scrollable::-webkit-scrollbar {
  width: 10px; /* Width of the scrollbar */
}
.scrollable::-webkit-scrollbar-thumb {
  background-color: #888; /* Color of the scrollbar thumb */
  border-radius: 5px; /* Rounded corners */
}
.scrollable::-webkit-scrollbar-thumb:hover {
  background-color: #555; /* Color on hover */
}
.scrollable::-webkit-scrollbar-track {
  background: #f1f1f1; /* Background of the scrollbar track */
}
```

SCREENSHOTS:

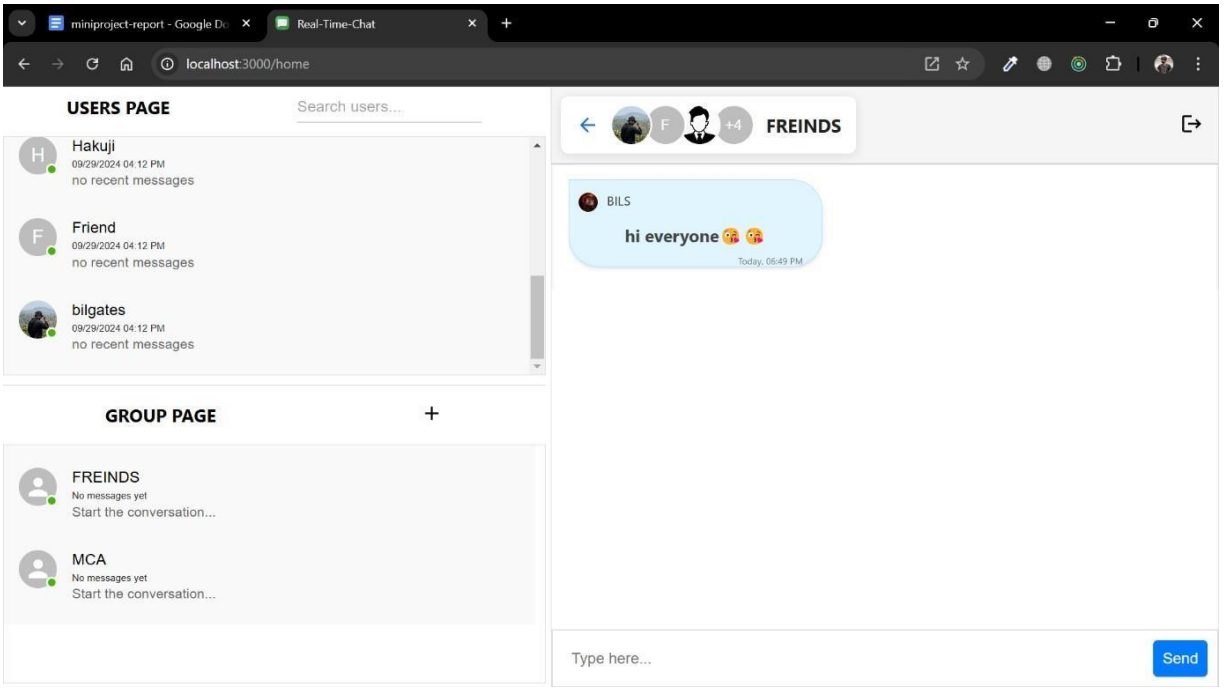
LOGIN PAGE:

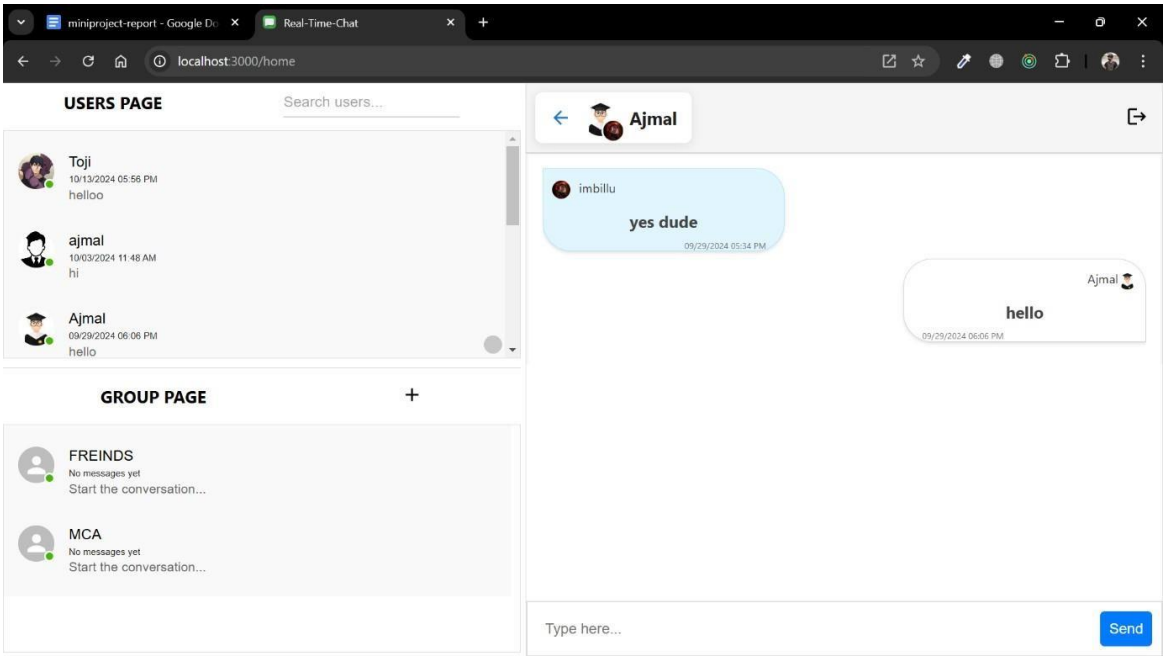


REGISTER USER:

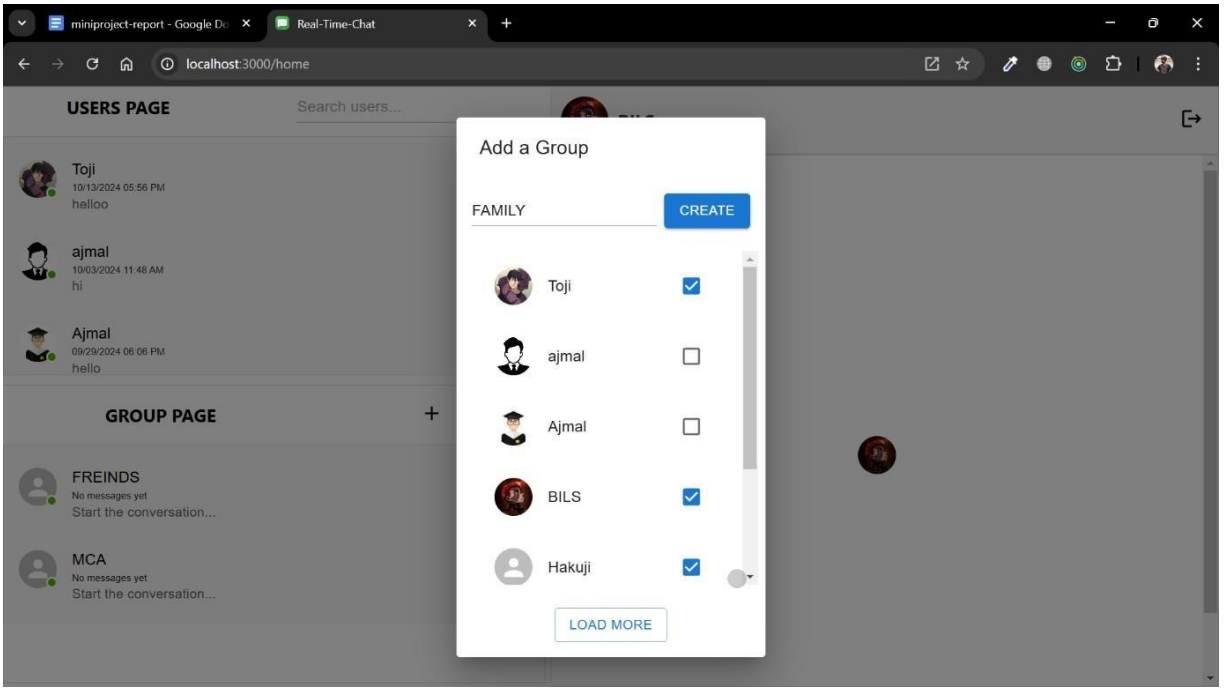


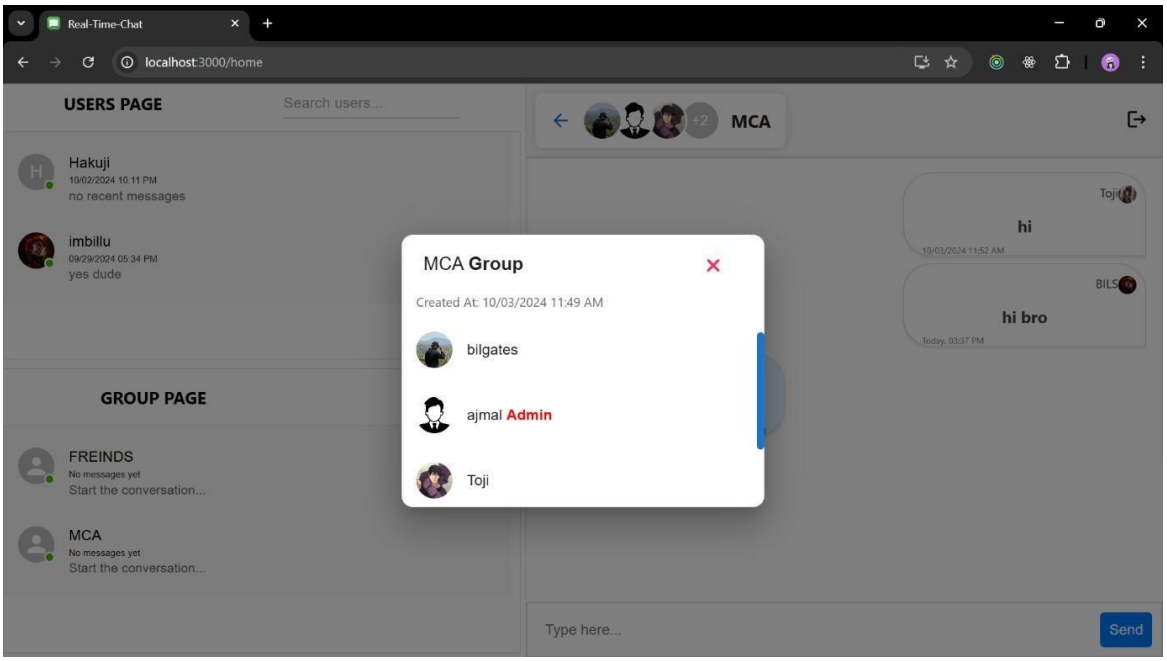
ONE TO ONE COMMUNICATION:



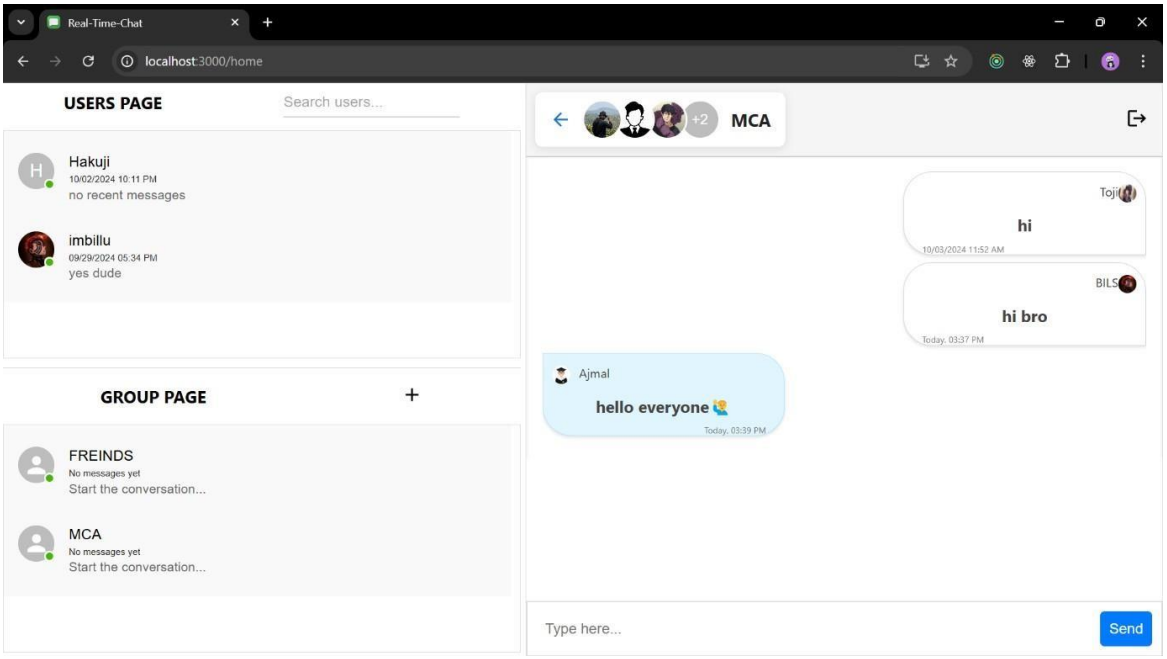


GROUP CREATION:





GROUP CHAT:


Real-Time-Chat
localhost:3000/home
USERS PAGE
Search users...
Hakuji
10/02/2024 10:11 PM
no recent messages
imbillu
09/29/2024 05:34 PM
yes dude
GROUP PAGE
FREINDS
No messages yet
Start the conversation...
MCA
No messages yet
Start the conversation...
+2
MCA
hi
10/03/2024 11:52 AM
hi bro
Today, 03:37 PM
Ajmal
hello everyone
Today, 03:39 PM
Type here...
Send